
UNIVERSITÉ DE SAINT ETIENNE // FACULTÉ DES SCIENCES

PROJET DE PROGRAMMATION ORIENTÉ OBJET
WARGAME

Rapport de projet

par Eyraud Mélanie // Goutelle Jérémy // Vandebussche Mathieu
le 7 décembre 2018

Tuteur enseignant :
Rémi EMONET

Sommaire

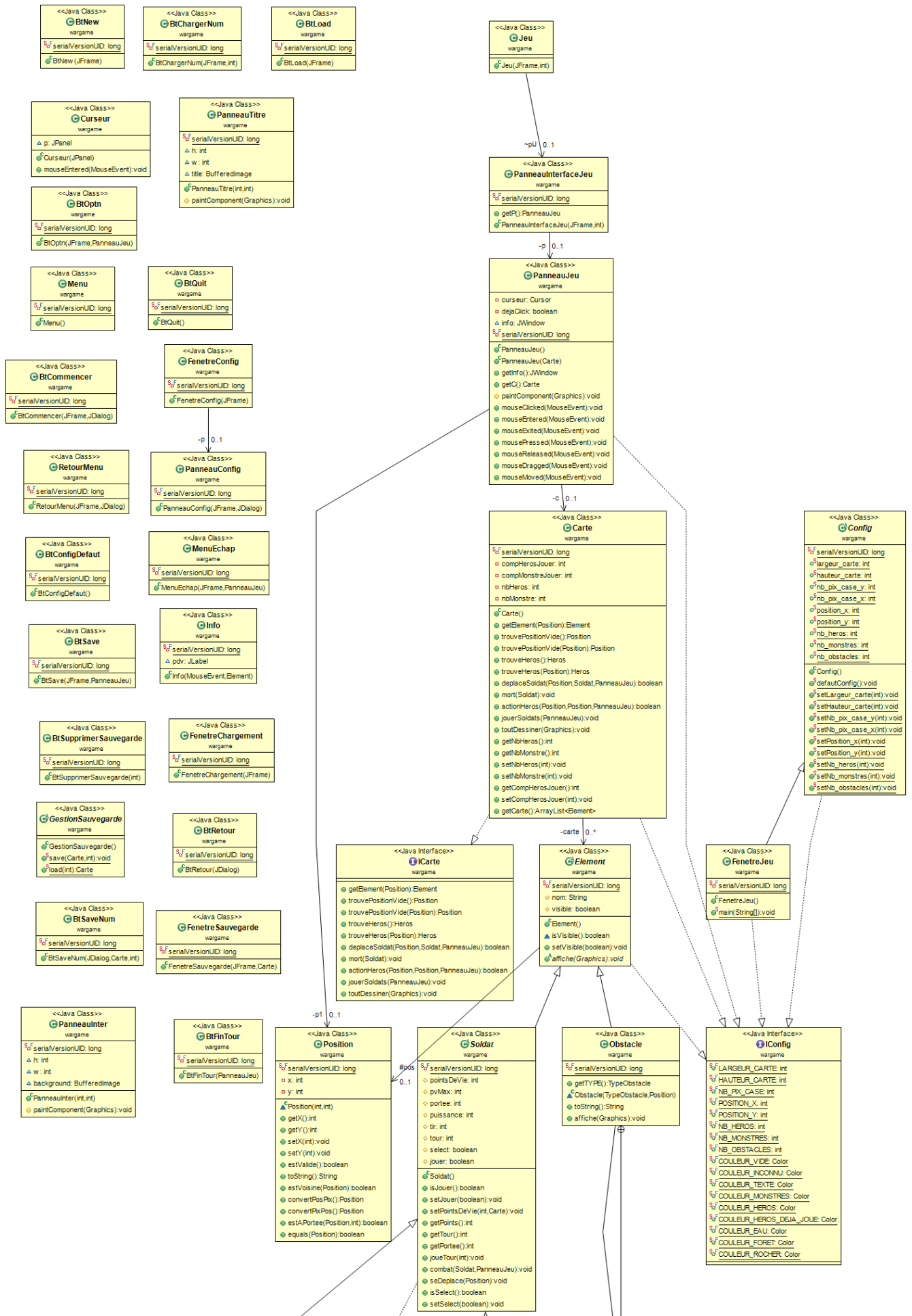
Introduction	2
1 Analyse du projet	3
2 Techniques	4
3 Présentation Synthétique	4
4 Estimation // Organisation	5
5 Liste Exhaustive des ressources externes et interne utilisé	5
Conclusion	6

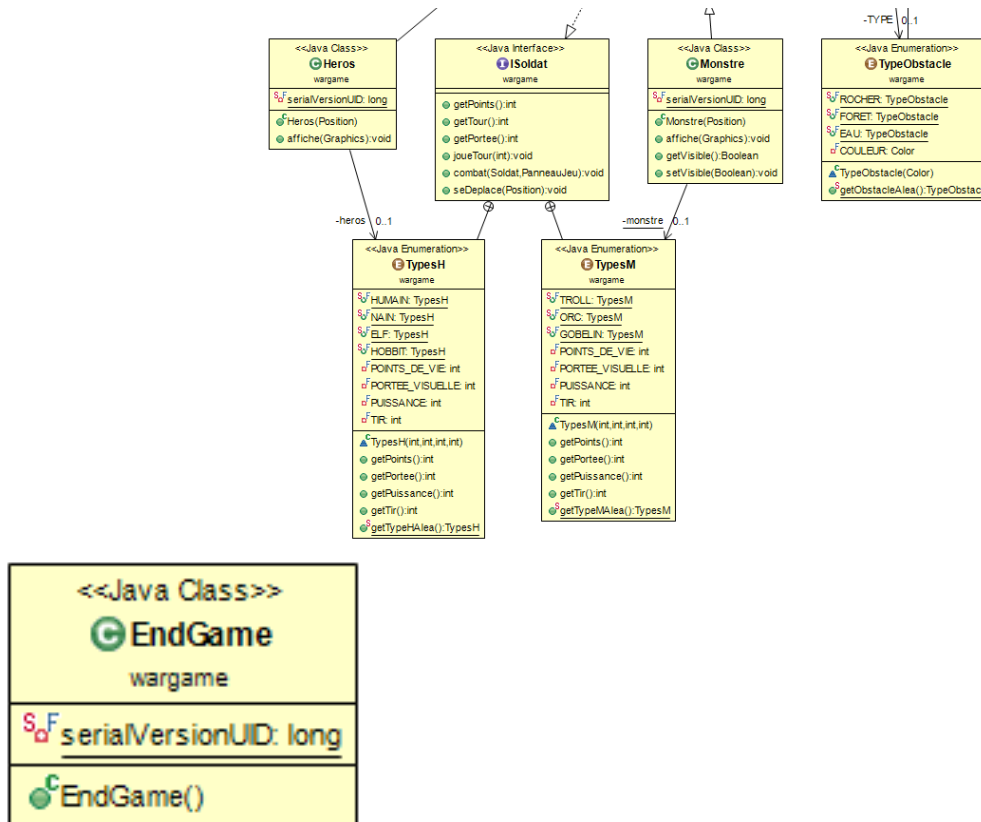
Introduction

Le projet de programmation orienté objet consiste à programmer en Java un jeu de stratégie prenant place dans le monde (imaginaire) du seigneur des anneaux. Pour cela nous devons utiliser les techniques de POO que nous avons vues en cours et celles que nous pourrions trouver sur différents forums et livres. Dans un premiers temps nous verrons comment nous avons gérés les classes du projet via un diagramme UML. Ensuite nous parlerons des techniques mises en place afin de gérer les classes entre elles (encapsulation, polymorphisme, héritage, exceptions, etc ...).

Par la suite l'organisation du travail sera présentés, puis une liste des sources utilisées et enfin nous conclurons sur ce que l'on a tiré de ce projet.

1 Analyse du projet





Les classes principales qui participent au bon fonctionnement du jeu sont liées comme énoncé dans le sujet du projet.

Mais les classes Bt... elles sont des classes qui héritent toutes de Jbuttons et ne sont liées à aucune classe. Il y a donc ici une amélioration possible en créant une interface Boutons qui regroupe toutes les classes qui s'occupent d'un type de bouton particulier.

La classe Menu est uniquement appelée lors du lancement du jeu et d'un click sur le bouton menu une fois la partie lancée.

En tout et pour tout, dans notre projet nous avons utilisé :

- 3 interfaces
- 27 classes
- 3 classes abstraites

2 Techniques

Les techniques que nous avons utilisées sont celles que l'on a appris en cours c'est à dire encapsulation, héritage et le polymorphisme. Nous avons aussi eu l'idée de mettre toutes les constantes importantes du jeu non pas dans les classes qui les utilisent mais dans la classe abstraite Config (cf le diagramme uml) et elle implemente IConfig pour les valeurs par défauts des héros pnj etc . Comme ceci on avait plus qu'à appeler les interfaces et on pouvait utiliser ces dites constantes.

3 Présentation Synthétique

Les fonctionnalités de notre jeux sont :

- Modification taille de la carte lors du lancement d'une nouvelle partie
- Possibilité de passer son tour
- Carte gérée par hexagone et non pas par case carrée (Plus de possibilité dans les déplacements)
- Le mode plein écran et son contenu s'adapte à n'importe quelle résolution d'écran

- Gestion de plusieurs sauvegardes et la possibilité de les supprimer en jeu

Pour entrer plus dans les détails, la modification de la taille de la carte est demandée a chaque lancement de la partie, et par conséquent lors de l’affichage de la carte de jeu cette dernière s’adapte a la taille du cadre en jeu. Bien évidemment la taille de la carte est enregistrée dans la sauvegarde.

La fonctionnalité de passer son tour à une utilité en plus de juste passer le tour du joueur, elle sert notamment a régénérer les PV des troupes du joueur.

La gestion de la carte en hexagone sert notamment a pouvoir se déplacer dans plus de direction qu’avec une gestion de la carte en "carré", les possibilités stratégiques sont donc plus nombreuses.

Comme par défaut le jeu se lance en plein écran, il fallait penser a adapter le jeu et son contenu à la taille de la fenêtre. Pour cela on a utilisé la bibliothèque dimension qui nous sert a récupérer la largeur et la hauteur de l’écran. On peut donc, grâce à la récupération de ces valeurs, adapter facilement la disposition et la taille des éléments de la fenêtre de jeu.

4 Estimation // Organisation

Le temps de travail de chacun a été :

- GOUTELLE Jérémy : 60-70 heures
- VANDENBUSSCHE Mathieu : 30-40 heures
- EYRAUD Mélanie : 10-20 heures

La répartition des tâches dans le projet est la suivante :

- GOUTELLE Jérémy : Fonctionnalité de base du jeu, gestion de la carte et tour de jeu.
- VANDENBUSSCHE Mathieu : Interface, création des visuels, rédaction du rapport.
- EYRAUD Mélanie : Gestion des soldats, clarification du code, création des commentaires, vérification de l’orthographe.

5 Liste Exhaustive des ressources externes et interne utilisé

L’image de fond pour le lancement du jeu vient du site <http://www.chainpixel.com/wallpaper/3591/>.

La grande majorité des techniques apprises viennent des cours de POO que l’on a acquises durant le semestre en cours et en TP.

Le reste des techniques viennent de tutos et explications variées des forums et sites "d’apprentissage" :

- Open Classroom : <https://openclassrooms.com/fr/> Forum et tuto en tout genre
- <https://www.developpez.net/forums/> Forum
- Stack Overflow : <https://stackoverflow.com/> Forum
- Eclipse JEE : Environnement de développement
- GIMP : création de visuel

Ces sites ont pour avantages d’être des forums, il y a donc des milliers de tutos et autres à disposition. Il est donc facile de trouver une aide ou une solution lorsque l’on a un problème ou autre. Les forums ont donc bien complété nos cours.

Par contre tous les visuels autres que l’image de fond du menu principal ont été créés par nous-même, c’est a dire bouton, personnage, pnj, curseur, etc.

Conclusion

Pour conclure, nous ne sommes pas mécontents du jeu que nous avons produit mais malgré tout nous avons tout de même quelques pistes d'améliorations en tête :

- IA : Une IA qui ne joue pas en mode aléatoire pour les déplacements et qui attaque un tant soit peu stratégiquement.
- Carte : Une génération de carte un peu meilleure, c'est à dire que la carte se génère en formant une rivière ou une chaîne de montagne. Car actuellement il y a seulement des lacs et quelques collines éparses rien de "cohérent".
- Visuel : Un meilleur aspect graphique pour plus facilement identifier quel personnage est de quelle "classe".

La programmation orientée objet a des avantages certains pour développer ce type d'application comparé au style de programmation "standard". Le principe de la POO qui repose sur les classes permet effectivement de faire de la programmation plus souple sans se préoccuper du passage de pointeur et de l'adressage.

La gestion des classes reste malgré tout assez difficile lorsque l'on est habitué au langage classique comme C.